

Nested Queries Practice Bank

SELECT, UPDATE, and DELETE queries with multiple nested subqueries in FROM, WHERE, and SELECT

Purpose: This document provides exam-level nested SQL query examples. The queries are intentionally layered so students can practice tracing subqueries step by step.

Practice Tables

Note: The queries below are based on the following sample tables. You can adjust names and values according to your own database.

Table 1: departments

dept_id	dept_name
10	IT
20	HR
30	Finance
40	Marketing

Table 2: employees

emp_id	emp_name	dept_id	salary	city
1	Ali	10	70000	Lahore
2	Sara	10	85000	Lahore
3	Ahmed	20	50000	Karachi
4	Hina	20	55000	Lahore
5	Bilal	30	90000	Islamabad
6	Ayesha	30	75000	Karachi
7	Zain	40	45000	Lahore
8	Noor	40	60000	Islamabad

9	Omar	10	65000	Karachi
10	Sana	30	90000	Lahore

Quick Concept Reminder

Concept	Meaning
Subquery in WHERE	Used to filter rows using the result of another query.
Subquery in FROM	Creates a temporary result table, also called a derived table.
Subquery in SELECT	Calculates an extra value to display with each row.
Correlated subquery	Inner query uses a value from the outer query.
UPDATE with subquery	Changes rows based on calculated values or filtered sets.
DELETE with subquery	Deletes rows based on nested conditions. It must be used carefully.

Part A: SELECT Queries with Multiple Nested Subqueries

Teacher note: These examples only retrieve data. They are safest for classroom practice.

Query 1:

```
SELECT e.emp_name, e.dept_id, e.salary
FROM employees e
WHERE e.dept_id IN (
  SELECT dept_avg.dept_id
  FROM (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept_id
  ) AS dept_avg
  WHERE dept_avg.avg_salary > (
    SELECT company_avg.avg_salary
    FROM (
      SELECT AVG(salary) AS avg_salary
      FROM employees
    ) AS company_avg
  )
);
```

Query 2:

```
SELECT e.emp_name, e.dept_id, e.salary
FROM employees e
WHERE e.salary > (
  SELECT dept_salary.avg_salary
  FROM (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept_id
  ) AS dept_salary
  WHERE dept_salary.dept_id = e.dept_id
);
```

Query 3:

```
SELECT e.emp_name, e.dept_id, e.salary
FROM employees e
WHERE e.dept_id IN (
  SELECT dept_count.dept_id
  FROM (
    SELECT dept_id, COUNT(*) AS total_employees
    FROM employees
    GROUP BY dept_id
  ) AS dept_count
  WHERE dept_count.total_employees > (
```

```

        SELECT min_required.total
      FROM (
        SELECT 2 AS total
      ) AS min_required
    )
  );

```

Query 4:

```

SELECT e.emp_name, e.dept_id, e.salary
FROM employees e
WHERE e.dept_id IN (
  SELECT high_total_dept.dept_id
  FROM (
    SELECT dept_id, SUM(salary) AS total_salary
    FROM employees
    GROUP BY dept_id
  ) AS high_total_dept
  WHERE high_total_dept.total_salary > (
    SELECT AVG(dept_totals.total_salary)
    FROM (
      SELECT dept_id, SUM(salary) AS total_salary
      FROM employees
      GROUP BY dept_id
    ) AS dept_totals
  )
);

```

Query 5:

```

SELECT e.emp_name, e.city, e.salary
FROM employees e
WHERE e.salary > (
  SELECT lahore_avg.avg_salary
  FROM (
    SELECT AVG(salary) AS avg_salary
    FROM employees
    WHERE city IN (
      SELECT city_table.city
      FROM (
        SELECT DISTINCT city
        FROM employees
        WHERE city = 'Lahore'
      ) AS city_table
    )
  ) AS lahore_avg
);

```

Query 6:

```
SELECT e.emp_name, e.dept_id, e.salary
FROM employees e
WHERE e.dept_id IN (
    SELECT dept_max.dept_id
    FROM (
        SELECT dept_id, MAX(salary) AS max_salary
        FROM employees
        GROUP BY dept_id
    ) AS dept_max
    WHERE dept_max.max_salary > (
        SELECT company_avg.avg_salary
        FROM (
            SELECT AVG(salary) AS avg_salary
            FROM employees
        ) AS company_avg
    )
);
```

Query 7:

```
SELECT e.emp_name, e.dept_id, e.salary
FROM employees e
WHERE e.salary IN (
    SELECT selected_dept_max.max_salary
    FROM (
        SELECT dept_id, MAX(salary) AS max_salary
        FROM employees
        WHERE dept_id IN (
            SELECT above_avg_depts.dept_id
            FROM (
                SELECT dept_id, AVG(salary) AS avg_salary
                FROM employees
                GROUP BY dept_id
            ) AS above_avg_depts
            WHERE above_avg_depts.avg_salary > (
                SELECT company_avg.avg_salary
                FROM (
                    SELECT AVG(salary) AS avg_salary
                    FROM employees
                ) AS company_avg
            )
        )
        GROUP BY dept_id
    ) AS selected_dept_max
);
```

Query 8:

```
SELECT d.dept_name
FROM departments d
WHERE d.dept_id IN (
  SELECT qualified_depts.dept_id
  FROM (
    SELECT dept_id
    FROM employees
    WHERE salary > (
      SELECT low_salary_avg.avg_salary
      FROM (
        SELECT AVG(salary) AS avg_salary
        FROM employees
        WHERE salary < (
          SELECT company_avg.avg_salary
          FROM (
            SELECT AVG(salary) AS avg_salary
            FROM employees
          ) AS company_avg
        )
      ) AS low_salary_avg
    )
  ) AS qualified_depts
);
```

Query 9:

```
SELECT e.emp_name, e.dept_id, e.salary
FROM employees e
WHERE e.dept_id IN (
  SELECT top_dept.dept_id
  FROM (
    SELECT dept_id, SUM(salary) AS total_salary
    FROM employees
    GROUP BY dept_id
  ) AS top_dept
WHERE top_dept.total_salary = (
  SELECT MAX(all_dept_totals.total_salary)
  FROM (
    SELECT dept_id, SUM(salary) AS total_salary
    FROM employees
    GROUP BY dept_id
  ) AS all_dept_totals
)
);
```

Query 10:

```
SELECT e.emp_name, e.dept_id, e.salary
FROM employees e
WHERE e.salary > (
    SELECT dept_avg.avg_salary
    FROM (
        SELECT dept_id, AVG(salary) AS avg_salary
        FROM employees
        GROUP BY dept_id
    ) AS dept_avg
    WHERE dept_avg.dept_id = e.dept_id
)
AND e.dept_id IN (
    SELECT high_avg_depts.dept_id
    FROM (
        SELECT dept_id, AVG(salary) AS avg_salary
        FROM employees
        GROUP BY dept_id
    ) AS high_avg_depts
    WHERE high_avg_depts.avg_salary > (
        SELECT company_avg.avg_salary
        FROM (
            SELECT AVG(salary) AS avg_salary
            FROM employees
        ) AS company_avg
    )
)
);
```

Query 11:

```
SELECT e.emp_name, e.dept_id, e.salary
FROM employees e
WHERE e.salary > (
    SELECT AVG(filtered_depts.avg_salary)
    FROM (
        SELECT dept_id, AVG(salary) AS avg_salary
        FROM employees
        WHERE dept_id IN (
            SELECT dept_count.dept_id
            FROM (
                SELECT dept_id, COUNT(*) AS total_employees
                FROM employees
                GROUP BY dept_id
            ) AS dept_count
            WHERE dept_count.total_employees > 2
        )
        GROUP BY dept_id
    ) AS filtered_depts
)
);
```

Query 12:

```
SELECT d.dept_name
FROM departments d
WHERE d.dept_id IN (
  SELECT dept_min.dept_id
  FROM (
    SELECT dept_id, MIN(salary) AS min_salary
    FROM employees
    GROUP BY dept_id
  ) AS dept_min
  WHERE dept_min.min_salary > (
    SELECT hr_avg.avg_salary
    FROM (
      SELECT AVG(salary) AS avg_salary
      FROM employees
      WHERE dept_id = (
        SELECT dept_id
        FROM departments
        WHERE dept_name = 'HR'
      )
    ) AS hr_avg
  )
);
```


Part B: UPDATE Queries with Multiple Nested Subqueries

Important Safety Note for UPDATE and DELETE

Do not run UPDATE or DELETE queries directly on original data. First run a SELECT version to preview affected rows. Use a duplicate practice table or transaction rollback. Each UPDATE and DELETE query below is written mainly for nested-query practice.

Query 1:

```
UPDATE employees e
SET salary = (
  SELECT final_result.new_salary
  FROM (
    SELECT emp_id, dept_avg_salary AS new_salary
    FROM (
      SELECT e1.emp_id, e1.salary, d.avg_salary AS dept_avg_salary
      FROM employees e1
      JOIN (
        SELECT dept_id, AVG(salary) AS avg_salary
        FROM employees
        GROUP BY dept_id
      ) AS d
      ON e1.dept_id = d.dept_id
    ) AS salary_details
    WHERE salary_details.salary < salary_details.dept_avg_salary
  ) AS final_result
  WHERE final_result.emp_id = e.emp_id
)
WHERE e.emp_id IN (
  SELECT eligible.emp_id
  FROM (
    SELECT e2.emp_id
    FROM employees e2
    JOIN (
      SELECT dept_id, AVG(salary) AS avg_salary
      FROM employees
      GROUP BY dept_id
    ) AS dept_avg
    ON e2.dept_id = dept_avg.dept_id
    WHERE e2.salary < dept_avg.avg_salary
  ) AS eligible
);
```

Query 2:

```
UPDATE employees e
SET e.salary = e.salary * 1.10
WHERE e.dept_id IN (
  SELECT selected_depts.dept_id
  FROM (
```

```

        SELECT dept_id, AVG(salary) AS avg_salary
        FROM employees
        GROUP BY dept_id
    ) AS selected_depts
WHERE selected_depts.avg_salary > (
    SELECT company_avg.avg_salary
    FROM (
        SELECT AVG(salary) AS avg_salary
        FROM employees
    ) AS company_avg
)
);

```

Query 3:

```

UPDATE employees e
SET e.salary = e.salary + 5000
WHERE e.salary < (
    SELECT company_avg.avg_salary
    FROM (
        SELECT AVG(salary) AS avg_salary
        FROM employees
    ) AS company_avg
)
AND e.dept_id IN (
    SELECT high_cost_depts.dept_id
    FROM (
        SELECT dept_id, SUM(salary) AS total_salary
        FROM employees
        GROUP BY dept_id
    ) AS high_cost_depts
WHERE high_cost_depts.total_salary > (
    SELECT AVG(dept_totals.total_salary)
    FROM (
        SELECT dept_id, SUM(salary) AS total_salary
        FROM employees
        GROUP BY dept_id
    ) AS dept_totals
)
);

```

Query 4:

```

UPDATE employees e
SET e.salary = (
    SELECT company_avg.avg_salary
    FROM (
        SELECT AVG(salary) AS avg_salary
        FROM employees
    ) AS company_avg
)

```

```

WHERE e.salary < (
    SELECT company_avg2.avg_salary
    FROM (
        SELECT AVG(salary) AS avg_salary
        FROM employees
    ) AS company_avg2
)
AND e.dept_id IN (
    SELECT dept_count.dept_id
    FROM (
        SELECT dept_id, COUNT(*) AS total_employees
        FROM employees
        GROUP BY dept_id
    ) AS dept_count
    WHERE dept_count.total_employees > 2
);

```

Query 5:

```

UPDATE employees e
SET e.salary = e.salary + 7000
WHERE e.salary IN (
    SELECT dept_toppers.max_salary
    FROM (
        SELECT dept_id, MAX(salary) AS max_salary
        FROM employees
        WHERE dept_id IN (
            SELECT high_avg_depts.dept_id
            FROM (
                SELECT dept_id, AVG(salary) AS avg_salary
                FROM employees
                GROUP BY dept_id
            ) AS high_avg_depts
            WHERE high_avg_depts.avg_salary > (
                SELECT company_avg.avg_salary
                FROM (
                    SELECT AVG(salary) AS avg_salary
                    FROM employees
                ) AS company_avg
            )
        )
        GROUP BY dept_id
    ) AS dept_toppers
);

```

Query 6:

```

UPDATE employees e
SET e.salary = e.salary * 0.97
WHERE e.dept_id IN (
    SELECT low_avg_dept.dept_id

```

```

FROM (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept_id
) AS low_avg_dept
WHERE low_avg_dept.avg_salary = (
    SELECT MIN(all_dept_avg.avg_salary)
    FROM (
        SELECT dept_id, AVG(salary) AS avg_salary
        FROM employees
        GROUP BY dept_id
    ) AS all_dept_avg
)
);

```

Query 7:

```

UPDATE employees e
SET e.salary = (
    SELECT dept_stats.max_salary
    FROM (
        SELECT dept_id, MAX(salary) AS max_salary
        FROM employees
        GROUP BY dept_id
    ) AS dept_stats
    WHERE dept_stats.dept_id = e.dept_id
)
WHERE e.salary < (
    SELECT dept_avg.avg_salary
    FROM (
        SELECT dept_id, AVG(salary) AS avg_salary
        FROM employees
        GROUP BY dept_id
    ) AS dept_avg
    WHERE dept_avg.dept_id = e.dept_id
);

```

Query 8:

```

UPDATE employees e
SET e.salary = e.salary + 3000
WHERE e.dept_id IN (
    SELECT selected_depts.dept_id
    FROM (
        SELECT dept_id, MIN(salary) AS min_salary
        FROM employees
        GROUP BY dept_id
    ) AS selected_depts
    WHERE selected_depts.min_salary > (
        SELECT hr_avg.avg_salary
        FROM (

```

```
        SELECT AVG(salary) AS avg_salary
        FROM employees
        WHERE dept_id = (
            SELECT dept_id
            FROM departments
            WHERE dept_name = 'HR'
        )
    ) AS hr_avg
);
```

Part C: DELETE Queries with Multiple Nested Subqueries

Important Safety Note for UPDATE and DELETE

Do not run UPDATE or DELETE queries directly on original data. First run a SELECT version to preview affected rows. Use a duplicate practice table or transaction rollback. Each UPDATE and DELETE query below is written mainly for nested-query practice.

Query 1:

```
DELETE FROM employees
WHERE emp_id IN (
  SELECT low_paid.emp_id
  FROM (
    SELECT emp_id
    FROM employees
    WHERE salary < (
      SELECT company_avg.avg_salary
      FROM (
        SELECT AVG(salary) AS avg_salary
        FROM employees
      ) AS company_avg
    )
  ) AS low_paid
);
```

Query 2:

```
DELETE FROM employees
WHERE dept_id IN (
  SELECT low_avg_depts.dept_id
  FROM (
    SELECT dept_id, AVG(salary) AS avg_salary
    FROM employees
    GROUP BY dept_id
  ) AS low_avg_depts
  WHERE low_avg_depts.avg_salary < (
    SELECT company_avg.avg_salary
    FROM (
      SELECT AVG(salary) AS avg_salary
      FROM employees
    ) AS company_avg
  )
);
```

Query 3:

```
DELETE FROM employees
WHERE emp_id IN (
  SELECT eligible.emp_id
```

```

FROM (
    SELECT e.emp_id
    FROM employees e
    JOIN (
        SELECT dept_id, AVG(salary) AS avg_salary
        FROM employees
        GROUP BY dept_id
    ) AS dept_avg
    ON e.dept_id = dept_avg.dept_id
    WHERE e.salary < dept_avg.avg_salary
) AS eligible
);

```

Query 4:

```

DELETE FROM employees
WHERE dept_id IN (
    SELECT lowest_total_dept.dept_id
    FROM (
        SELECT dept_id, SUM(salary) AS total_salary
        FROM employees
        GROUP BY dept_id
    ) AS lowest_total_dept
    WHERE lowest_total_dept.total_salary = (
        SELECT MIN(all_totals.total_salary)
        FROM (
            SELECT dept_id, SUM(salary) AS total_salary
            FROM employees
            GROUP BY dept_id
        ) AS all_totals
    )
);

```

Query 5:

```

DELETE FROM employees
WHERE dept_id IN (
    SELECT single_emp_depts.dept_id
    FROM (
        SELECT dept_id, COUNT(*) AS total_employees
        FROM employees
        GROUP BY dept_id
    ) AS single_emp_depts
    WHERE single_emp_depts.total_employees = (
        SELECT required_count.total
        FROM (
            SELECT 1 AS total
        ) AS required_count
    )
);

```

Query 6:

```
DELETE FROM employees
WHERE emp_id IN (
  SELECT selected_employees.emp_id
  FROM (
    SELECT emp_id
    FROM employees
    WHERE salary < (
      SELECT lahore_avg.avg_salary
      FROM (
        SELECT AVG(salary) AS avg_salary
        FROM employees
        WHERE city IN (
          SELECT city_table.city
          FROM (
            SELECT DISTINCT city
            FROM employees
            WHERE city = 'Lahore'
          ) AS city_table
        ) AS lahore_avg
      ) AS selected_employees
    )
  );
```

Query 7:

```
DELETE FROM employees
WHERE emp_id IN (
  SELECT removable.emp_id
  FROM (
    SELECT e.emp_id
    FROM employees e
    JOIN (
      SELECT dept_id, MAX(salary) AS max_salary
      FROM employees
      GROUP BY dept_id
    ) AS dept_max
    ON e.dept_id = dept_max.dept_id
    WHERE e.salary < dept_max.max_salary
    AND e.dept_id IN (
      SELECT low_avg.dept_id
      FROM (
        SELECT dept_id, AVG(salary) AS avg_salary
        FROM employees
        GROUP BY dept_id
      ) AS low_avg
      WHERE low_avg.avg_salary < (
        SELECT company_avg.avg_salary
```



```

        FROM (
            SELECT AVG(salary) AS avg_salary
            FROM employees
        ) AS company_avg
    )
) AS removable
);

```

Query 8:

```

DELETE FROM employees
WHERE dept_id IN (
    SELECT weak_depts.dept_id
    FROM (
        SELECT dept_id, MAX(salary) AS max_salary
        FROM employees
        GROUP BY dept_id
    ) AS weak_depts
    WHERE weak_depts.max_salary < (
        SELECT company_avg.avg_salary
        FROM (
            SELECT AVG(salary) AS avg_salary
            FROM employees
        ) AS company_avg
    )
);

```

Recommended Classroom Activity

Step	Task
Step 1	Ask students to identify the innermost subquery.
Step 2	Ask them to write the output of the innermost query first.
Step 3	Move one level outward and replace the subquery with its result.
Step 4	For UPDATE and DELETE, ask students to write a SELECT preview query first.
Step 5	Only after dry-run analysis, discuss what rows will be updated or deleted.